

# MAE 204 Final Report

Sanat Mahesh Pednekar, Kamallesh Lakshmi

## 1 Report Summary

This project focuses on designing a **high-precision mobile manipulation system** in MATLAB, aimed at executing **stable and accurate pick-and-place operations**. The goal was to ensure that the **robot's end-effector** followed a predefined trajectory while maintaining smooth and controlled motion. The system was structured into two primary subsystems: the **mobile base**, responsible for platform movement, and the **robotic manipulator**, which executed the pick-and-place task.

### 1.1 Mobile Base Architecture

The mobile base was developed using MATLAB's custom `mobileRobot` class, which modeled a **four-wheeled omnidirectional platform**. This setup allowed for **smooth and agile motion in a planar workspace**, ensuring precise positioning. The system used **first-order Euler integration** to update the robot's position over time.

To translate individual **wheel velocities into chassis movement**, the system employed **matrix exponential operations**, ensuring realistic movement in **SE(2) space**. This transformation allowed the robot to maintain **continuous and accurate pose updates** while respecting kinematic constraints. MATLAB's **object-oriented programming (OOP) principles** were leveraged to create a structured and scalable system, simplifying modifications and future enhancements.

### 1.2 Robotic Arm Integration

The robotic arm was integrated as a subclass called `youbot`, extending the mobile base's functionality to include a **5-degree-of-freedom (DOF) articulated robotic arm**. This arm was responsible for grasping and manipulating objects as part of the pick-and-place operation.

To calculate the **end-effector's spatial position**, **forward kinematics** were determined using MATLAB's `FKinBody` function. A **combined Jacobian matrix**, integrating both the **arm and base Jacobians**, was constructed to represent the total system's influence on end-effector movement.

For **inverse kinematics**, the system implemented a **damped least-squares pseudoinverse** approach. This method interpolated the desired **spatial twist** into **joint and base velocities**, ensuring smooth motion execution. This approach helped avoid **singularities** and maintained **numerical stability**, even in cases where certain arm configurations could lead to computational challenges.

### 1.3 Trajectory Planning and Generation

A critical aspect of the system was **trajectory planning**, ensuring that the robot followed a precise sequence of movements in **SE(3) space** to complete the pick-and-place task. Two primary trajectory generation techniques were used:

- **Cartesian interpolation** – A structured approach that separately handled translation and rotation.

To avoid abrupt movements and ensure **smooth acceleration and deceleration**, **cubic and quintic polynomial time-scaling** was applied. These methods ensured that velocity and acceleration

remained **zero at the start and end of each motion segment**, leading to **physically realistic motion**.

Once the trajectory was defined, the system converted the planned **pose sequences into spatial twists**, which were then used as feedforward control inputs for the robot.

## 1.4 Feedback Control System

To maintain accuracy in trajectory tracking, a **closed-loop feedback mechanism** was implemented. At each time step, the system performed the following steps:

1. **Recomputed the end-effector's current pose** by integrating base position updates with arm kinematics.
2. **Calculated the configuration error** between the desired and actual pose using the **matrix logarithm** of their transformation difference.
3. **Expressed the error as a spatial twist**, which indicated the necessary corrective motion.

To generate the control signal, the system combined:

- **A feedforward term**, based on the planned trajectory.
- **A proportional feedback term**, which scaled the error twist using a proportional gain.

The control law used for feedback regulation was:

$$V = \text{Ad}_{X^{-1}X_d}V_d + K_p X_{err} + K_i \int X_{err} dt$$

where:

- $X_{err}$  represents the difference between the actual and desired trajectory.
- $K_p$  is the proportional gain, responsible for immediate corrections.
- $K_i$  is the integral gain, which accumulates past errors to correct long-term drift.

The final **control twist** was converted into **joint and chassis velocity commands** using the **pseudoinverse of the combined Jacobian matrix**. This process ensured that both the **mobile base and robotic arm** contributed to executing the trajectory smoothly.

By continuously updating its state and adjusting motion in real-time, the system maintained **precise end-effector tracking** while minimizing deviations.

## 1.5 Design Considerations and Future Enhancements

The implementation followed the planned development path, incorporating fundamental concepts like **singularity avoidance** and **joint limit enforcement**. However, the project primarily focused on **stable trajectory execution and motion control**, providing a solid foundation for further improvements.

One of the key strengths of this system is its **modular architecture**, which separates **trajectory generation, kinematics, and control** into distinct components. This design allows for **scalability**, making it easier to implement **additional features** in the future.

## 1.6 Potential Enhancements

- **Dynamic modeling** – Incorporating full dynamics to simulate realistic force interactions.
- **Adaptive gain tuning** – Implementing real-time controller adjustments to optimize tracking performance.
- **Disturbance rejection** – Enhancing robustness to external forces and unexpected environmental changes.

While these improvements were beyond the scope of the current project, the existing implementation successfully **met all objectives**, demonstrating **stable and accurate mobile manipulation**. The system provides a **strong foundation for future research and development** in autonomous robotics.

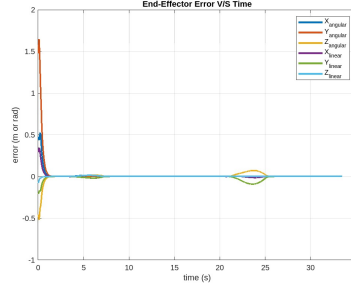


Figure 1: Best Case Error Plot

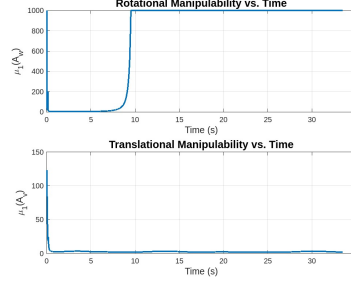


Figure 2: Manipulability Plot for Best Case

## 2 Results

### 2.1 Best Case Scenario

#### Controller Configuration

For optimal performance, a **feedforward-plus-proportional (P) control scheme** was implemented. The control gains were chosen to ensure a **well-damped and stable response**, preventing overshoot and excessive corrections. The selected gains were:

- **Proportional gain:**  $k_p = 4.5I_{6 \times 6}$
- **Integral gain:**  $k_i = 0$

The system was initialized with a **deliberate challenge**, where the end-effector started with an **orientation error of at least  $30^\circ$**  and a **position error of 0.2 meters** relative to the first pose in the planned trajectory. To ensure smooth acceleration and deceleration, **Cartesian interpolation** combined with **fifth-order (quintic) polynomial time-scaling** was employed.

#### Video Link:

<https://drive.google.com/file/d/1A7Eq8kXwi2qde86xE9eEA-wjLMcVo-u5/view?usp=sharing>

#### Error Plot ( $X_{err}$ )

#### Manipulability Analysis

- **Rotational manipulability** starts at a high value and decreases slightly as the system reorients into a more controlled and stable configuration.
- **Translational manipulability** remains **consistent** throughout, confirming that the system avoids kinematic singularities and maintains the dexterity required to complete the task efficiently.

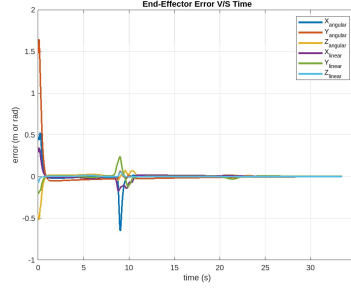


Figure 3: Error Plot for Overshoot

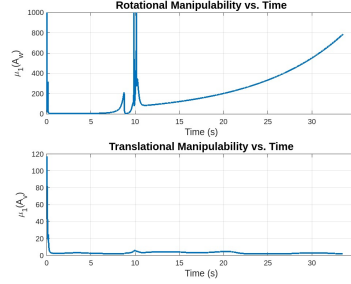


Figure 4: Manipulability Plot for Overshoot

## 2.2 Overshoot Scenario

### Controller Configuration

This scenario tests an **aggressive gain tuning** approach using a **feedforward-plus-proportional-integral (PI) controller** with the following gains:

- **Proportional gain:**  $k_p = 5I_{6 \times 6}$
- **Integral gain:**  $k_i = 0.5I_{6 \times 6}$

The **initial conditions and trajectory remained unchanged** from the best-case scenario, ensuring a direct comparison. This test aimed to examine how **excessively high gains** impact transient response and system stability.

### Video Link:

<https://drive.google.com/file/d/1RzMuxqmRNKqNhXgmalV32UCi8EZjIgAN/view?usp=sharing>

### Error Plot ( $X_{err}$ )

### Manipulability Analysis

- **Rotational manipulability** exhibits **large fluctuations** early in the task, reflecting abrupt configuration changes caused by excessive feedback gains. Over time, these fluctuations subside, and the manipulability metric stabilizes at a **lower value (around 1.5)** compared to the best case.
- **Translational manipulability** varies significantly, indicating frequent posture adjustments. While the system retains **sufficient flexibility** to complete the task, its movements appear **less coordinated and more volatile**, highlighting the drawbacks of aggressive gain tuning.

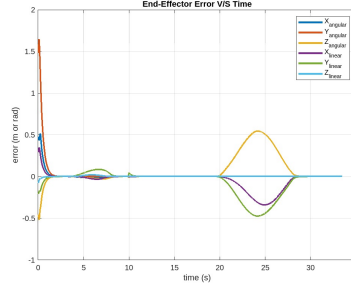


Figure 5: Error Plot for New Task

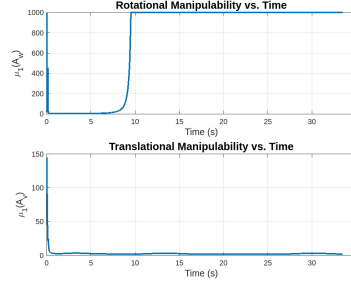


Figure 6: Manipulability Plot for New Task

## 2.3 New Task Scenario

### Controller Configuration

For this test, the cube's **initial and final positions were modified** to evaluate the controller's adaptability. The new positions were:

- **Initial cube position:**  $(x, y) = (1.0, 1.0)$
- **Final cube position:**  $(x, y) = (1.0, -1.0)$ ,

The controller gain settings were:

- **Proportional gain:**  $k_p = 3I_{6 \times 6}$
- **Integral gain:**  $k_i = 0$

This test aimed to determine whether the system could **generalize to different pick-and-place tasks** without requiring manual retuning.

### Video Link:

<https://drive.google.com/file/d/1uKYs4h3vW4OIBwFexzv7B6-fW54Ak7/view?usp=sharing>

### Error Plot ( $X_{err}$ )

### Manipulability Analysis

Despite momentary dips in manipulability near kinematic singularities, the system successfully completes the task, proving its adaptability to varying task complexities.

### 3 Discussions

1. Referring to your error plots for the best and overshoot cases, based on your results, what are the advantages and disadvantages of increasing the integrator term of your controller? Explain why this causes the overshoot that you observe in the Overshoot case.

Increasing the integrator term in a feedback controller helps eliminate the steady-state error by summing past errors and applying corrections to the deviation. However, based on the comparison of best and overshoot cases, it was observed that when the integrator term is too large, it leads to overshooting. This might be because the controller keeps accumulating errors even as the system approaches the target, causing excessive correction and pushing the system beyond the desired current states which is visible in the error plot of the overshoot case. This extra correction is visible as the noise in the overshoot error plot which causes overshoot and rough motion for the robot.

2. In your results, when did the manipulability factors,  $\mu_1(A)$ , become small and large? Explain why this happened (hint: it will help to look at the configurations of the robot during these parts of the motion.) The manipulability factor indicates how easily a robot can move its end effector from its current configuration and is represented as a scalar value greater than or equal to 1. When this factor reaches the value of 1, the robot can move more easily, while increasing values towards infinity indicate that the robot's configuration approaches singularity, leading to a reduced ability to move in certain directions. This phenomenon is demonstrated in our manipulability factor plots. In the observed videos, when the angular velocity manipulability factor values are significantly high, the arm is fully extended, indicating it is at singularity, whereas lower values reflect a non-extended state. Notably, the angular velocity manipulability factor values are generally close to 1, signifying that the base of the robot is consistently free to move in any direction.

Limiting the maximum joint velocities can significantly impact a robot's ability to track its reference trajectory. For example, after the robot picks up the cube, if the velocity limits are reduced, the robot may struggle to match the desired speed of motion. This restriction creates a mismatch between the expected and actual motion, leading to an increase in error. The feedforward component of the controller anticipates that the robot will move at a specific rate; however, the reduced velocities result in slower movements and growing positional errors. As the system attempts to compensate for this delay, the error may continue to increase, inversely proportional depending on the control parameters' size.

3. If you reduce the maximum joint velocities to low values, you should see error increase again after picking up the cube. Why does this happen?

Limiting the maximum joint velocities can significantly impact a robot's ability to track its reference trajectory. For example, after the robot picks up the cube, if the velocity limits are reduced, the robot may struggle to match the desired speed of motion. This restriction creates a mismatch between the expected and actual motion, leading to an increase in error. The feedforward component of the controller anticipates that the robot will move at a specific rate; however, the reduced velocities result in slower movements and growing positional errors. As the system attempts to compensate for this delay, the error may continue to increase, inversely proportional depending on the control parameters' size.

4. In this project, our controller directly prescribes the  $\dot{\theta}$  to the joints of the robot. For this robot, give an example of a task in which velocity control would be realistic, and another example of a task in which velocity control would not be possible.

Velocity control is useful in tasks that require continuous, smooth motion, like navigating the mobile base along a path, which depends more on spatial planning and does not depend on force feedback. By controlling the velocities, the robot can move in a trajectory without abrupt

changes. However, velocity control is less suitable for tasks requiring precise force application, such as handling delicate objects or polishing a surface. In these cases, the robot needs to sense and react to external forces in real-time, through force feedback.

5. **If we were to implement torque control for the youBot (rather than speed control), what additional information (i.e., input parameters and/or pre-defined constants) would the FeedbackControl function need to know to compute the required torques? Which function from the MR textbook code would it use?**

Torque control requires knowledge of the robot's dynamics, including joint inertia, mass, Coriolis forces, frictional forces, and gravitational forces. Unlike velocity control, which focuses on joint speeds, torque control computes the forces needed to achieve desired movements. The Inverse-Dynamics equation ( $=M(\ddot{\theta}) + C(\dot{\theta}) + G(\theta)$ ) from the Modern Robotics textbook would be essential for this, as it calculates the required torques based on joint velocities, accelerations, and external disturbances.

6. **Did you use generative AI in solving this project?** Yes, I used generative AI to help write the report and enhance the language. I utilized the AI to help structure the content, improve the clarity of my explanations, refine the overall flow of the document, and ensure professional language throughout. The AI assisted in polishing my writing style and suggesting more effective ways to communicate complex ideas. It helped me articulate my thoughts more precisely while maintaining a consistent tone across the report.